

Python Data Structure – List

串列 or 陣列

Cheng-Te Li (李政德)

chengte@mail.ncku.edu.tw

Dept. of Statistics, NCKU



Course Info (11/13)

- **Announcements**
 - **Midterm Exam: 11/19**
 - **HW3: 11/26**
 - **Programming Exam 2: 11/27**
- **Today & Next Week**
 - **Lecture 5: List**
 - **Lecture 6: File Processing**

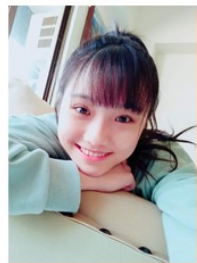
HW3

Deadline
11/25 (Monday)
23:59

- **Homework 3 (HW3)** is announced
 - String, if-elif-else, loops, **list**
 - **4 Problems**: 1st: 離開圓桌, 2nd: 多項式運算, 3rd: **Connect Four**, 4th: 矩陣數字取代
 - More logic thinking than Homework 2
 - Please start writing HW3 as early as possible

12 Teaching Assistants

- Make use of TA times



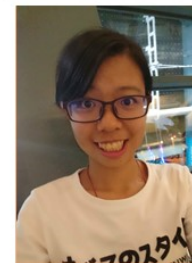
李艾霓
統計二



劉米婷
統計二



詹永裕
統計三



林佳霈
統計三



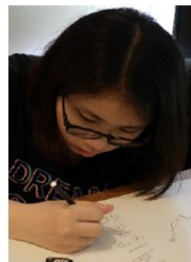
陳冠穎
統計四



馬少倫
統計四



張瑜婕
統計四



王珮綺
統計四



陳佑嘉
統計四



范凱翔
碩一



蕭少平
碩一



蔡昌銘
碩一

Midterm (20%)

- **11/19** (Tuesday): Midterm Exam (期中考)
- 09:10am – 12:00pm (寫完交卷即可離開)
- 紙本作答
- 題型(類似hw1_p1, hw2_p1)，可能包含：
選擇、寫出print出什麼、填空、除錯、紙上寫程式
- NO Open book, NO slides, NO paper materials
- NO Computer, NO Internet
- Coverage: Lecture 01~06 (till 11/12)
 - Homework 1-3, Programming Exam 1
- Many problems vary from or **the same** as slides or HWs

Programming Exam 2 (10%)

- **11/26** (Tuesday): Programming Exam 2
(第2次上機程式考試) for each student (not each team)
- Open book, slides, and paper-based materials
- NO Internet (except for submission on Moodle)
- You can use either PC here or your own laptop
- Start from **09:10am to 12:00pm**
- Coverage: Lectures till **11/12**
 - String, if-elif-else, loops: while, List
 - Homework 1, 2, and 3

How to Learn Coding for Very Beginners?

- **Write by yourself 動手實作!**
 - 把投影片的題目蓋起來，自己動手寫一次
 - Exercises, Homeworks, Quizzes
- More discussion → 分組，跟組員或助教討論
- Read others' codes → 補充閱讀
- Write comments for your codes
(to make sure you truly understand the code)
- When you codes have errors or unexpected results ...
把變數print出來看!
- **Or debug by asking Google** (你的錯誤訊息肯定有人犯過)
(what does the error message mean?)
- Improve your English
(many coding documents are English)

Data Collection

- Up to now, we have put a single value into a single variable. Ex: num = 1, day = "Tuesday"
- But what if we want to work with collections of data?

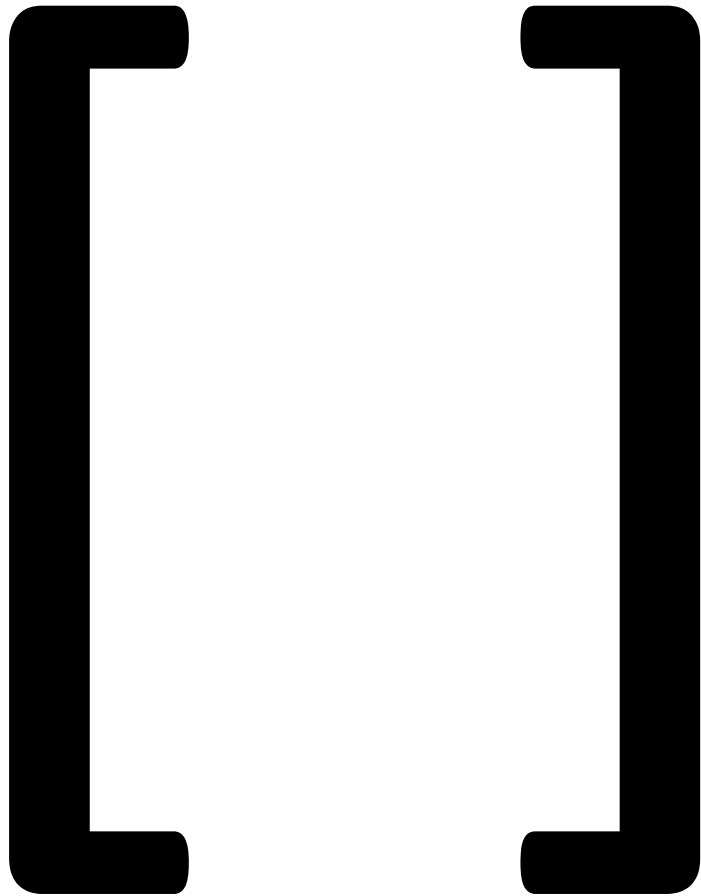
Example

The number of whales sighted near a research station, counted each day for a two-week period, is given in the next table.

Day:	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Whales:	5	4	7	3	2	3	2	6	4	2	1	7	1	3

- Without a collection data type, we need **fourteen** variables to store these values
- This is still manageable, but what if have data for a year, or a decade, or a century?

Lists



Versatile (實用)
Incredibly common

Lists

Example

The number of whales sighted near a research station, counted each day for a two-week period, is given in the next table.

Day:	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Whales:	5	4	7	3	2	3	2	6	4	2	1	7	1	3

- Solution: Use a **list**
 - Put the values, separated by **commas**, inside **square brackets []**

```
>>> [5, 4, 7, 3, 2, 3, 2, 6, 4, 2, 1, 7, 1, 3]  
[5, 4, 7, 3, 2, 3, 2, 6, 4, 2, 1, 7, 1, 3]
```

- **list** is a Python data type
- Therefore, we can have objects of lists
- A list can be assigned to a variable and it owns functions
- There are standard functions that operate on lists

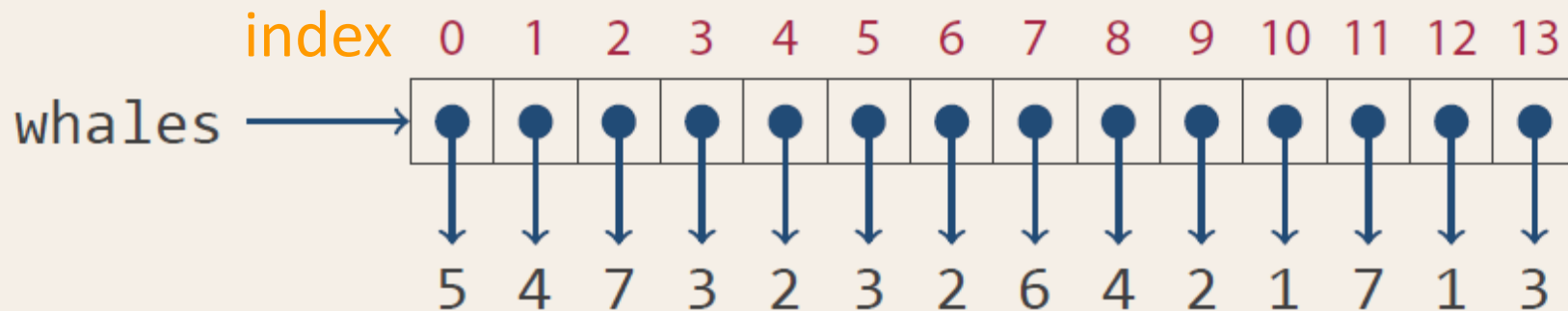
List Assignment

Example (Assign a list to a variable)

```
>>> whales = [5, 4, 7, 3, 2, 3, 2, 6, 4, 2, 1, 7, 1, 3]
>>> whales
[5, 4, 7, 3, 2, 3, 2, 6, 4, 2, 1, 7, 1, 3]
```

- The variable `whales` contains a **reference** to the list
- Each of the **indices** in the list contains a reference to a number

List Memory Model



List Indexing

- Think of a **list** as a **vector**: Just as we can **index** over the vector components, we **index** over the elements in a list
- The first index is **0**: Think of this as saying
 - “We are **x positions from the front**” Or:
 - “There are **x elements before this one**”
- To refer to a particular item, put the index in brackets after a reference to the list, such as the name of a variable

```
whales = [5, 4, 7, 3, 2, 3, 2, 6, 4, 2, 1, 7, 1, 3]
whales[0]    # 5
whales[11]   # 7
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29][5]    # 13
```

Legal Indices

- For a list of length n , a legal index i is an integer in the set $\{0 \leq i \leq n - 1\}$
- Trying to use an out-of-range index is an error

Indexing out of range

```
>>> whales = [5, 4, 7, 3, 2, 3, 2, 6, 4, 2, 1, 7, 1, 3]
>>> whales[23]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range
```

Indexing Backwards

- Python also lets us index backwards from the end of a list
- Index **-1** refers to **the last item**, and so on
- We can also assign the values in a list to other variables

```
whales = [5, 4, 7, 3, 2, 3, 2, 6, 4, 2, 1, 7, 1, 3]
whales[-1]      # 3
whales[-14]     # 5
third = whales[2]
print("On the third day", third, "whales were seen.")
# On the third day 7 whales were seen.
```

The Empty List

- Just as **0** for arithmetic, and the empty string `""` for strings
- There is also an empty list, written `[]`, with no elements
- An empty list has no legal indices

```
whales = []      # the empty list
```

```
whales[0]
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
IndexError: list index out of range
```

```
whales[-1]
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
IndexError: list index out of range
```

Lists are Heterogeneous

- Lists can contain any type of data
- We can also “mix” different data types in one list
- Therefore, lists are called heterogeneous

```
data = ["Jeremy Lin", "JLin7", 191.5, 91]  
print(data[0], ", the basketball star of tomorrow. His  
information includes", data[2], "cm and", data[3], "kg")
```

- Using a list to aggregate data is not a good idea
- Besides, many list functions assume all items have the same type, and they fail if they don't

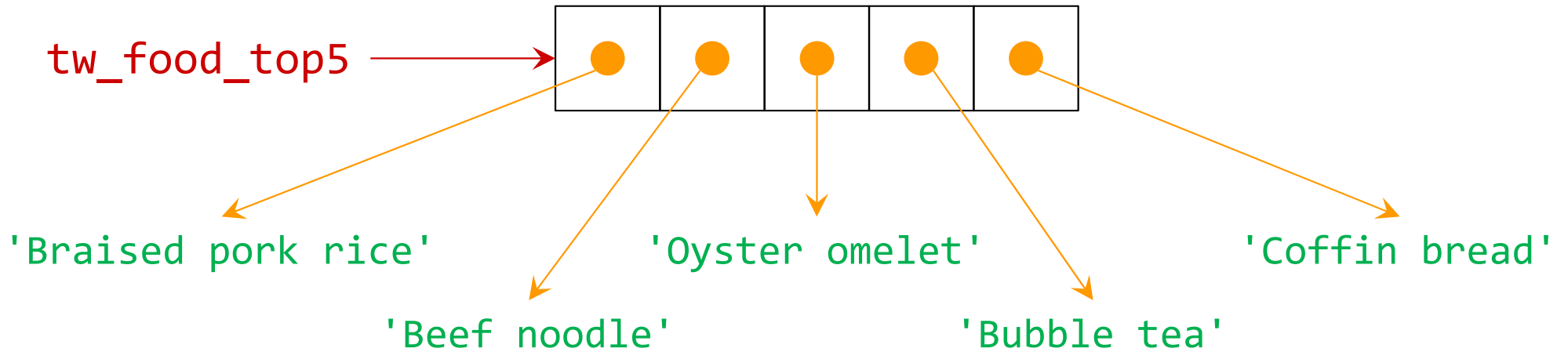
Lists are mutable

- We can modify lists!
- We can modify a list after it has been declared

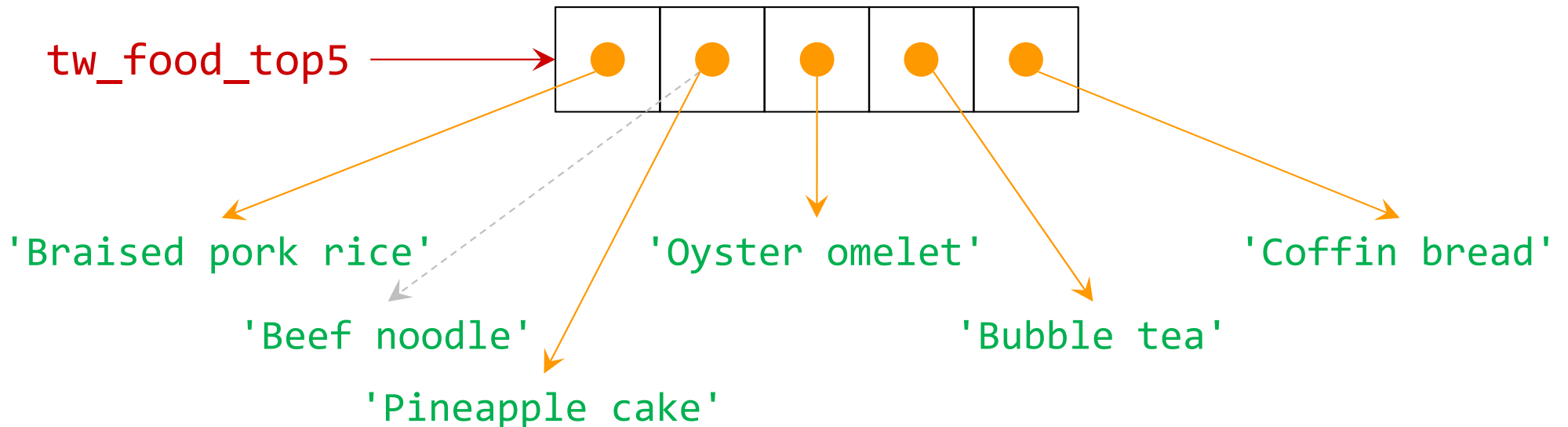
```
tw_food_top5 = ['Braised pork rice', 'Beef noodle', 'Oyster omelet', 'Bubble tea', 'Coffin bread']  
print(tw_food_top5)  
# ['Braised pork rice', 'Beef noodle', 'Oyster omelet',  
  'Bubble tea', 'Coffin bread']  
tw_food_top5[1] = 'Pineapple cake'  
print(tw_food_top5)  
# ['Braised pork rice', 'Pineapple cake', 'Oyster omelet',  
  'Bubble tea', 'Coffin bread']
```


List Mutation

Before mutation



After mutation



Strings are Immutable

- The expression `L[i]`, where `L` is a list and `i` an index, behaves just like a normal variable
 - “Get the value of the item at `i` in `L`”
 - “Figure out where item `i` is in `L` so that we can overwrite it”
- Compare this to strings, where we cannot change a letter after the string has been created
- For example, `title()` actually **creates a new string**

```
name1 = "Jeremy lin"
```

```
name1[7] = "L" →
```

```
name2 = name1.title()
```

```
print(name1, "\t", name2) # Jeremy lin      Jeremy Lin
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: 'str' object does not support item assignment
```

Built-In List Functions

Table: Built-in list functions

Function	Description
<code>len(L)</code>	Returns the number of items in list <code>L</code>
<code>max(L)</code>	Returns the maximum value in list <code>L</code>
<code>min(L)</code>	Returns the minimum value in list <code>L</code>
<code>sum(L)</code>	Returns the sum of the values in list <code>L</code>

- Note that we have seen some of these functions before
- For example, `len()` has previously been applied to `strings`

List Functions and Range Checking

```
jump_water = [81.5, 67.8, 113, 99.2, 104.7]
len(jump_water)    # 5
max(jump_water)    # 113
min(jump_water)    # 67.8
sum(jump_water)    # 446.8
print("average score =", sum(jump_water)/ len(jump_water)) # 93.24

i = 2
# check whether an index is in a range
0 <= i < len(jump_water)    # True
```

```
1 jump_water = [81.5, 67.8, 113, 99.2, 104.7]
2 i = 0
3 total = 0 # initialization
4 while i < len(jump_water):
5     total += jump_water[i]
6     i = i + 1
7 print("average score =", total/len(jump_water))
```

if no initialization

```
Traceback (most recent call last):
  File "test.py", line 5, in <module>
    total += jump_water[i]
NameError: name 'total' is not defined
```

Example

- Use **lists** and **while** loop to calculate and print the average scores of three students, whose names and scores of Chinese, English, and Math are given in lists

```
1 students = ['Tom', 'Mary', "John"]
2 chinese = [90, 80, 70]
3 english = [69, 70, 89]
4 math = [88, 81, 79]
5 i = 0
6 average = [0, 0, 0]
7 while i < len(students):
8     average[i] = chinese[i] + english[i] + math[i]
9     average[i] = average[i] / 3
10    print("The average score for", students[i], "is", average[i])
11    i = i + 1
```

```
The average score for Tom is 82.33333333333333
The average score for Mary is 77.0
The average score for John is 79.33333333333333
```

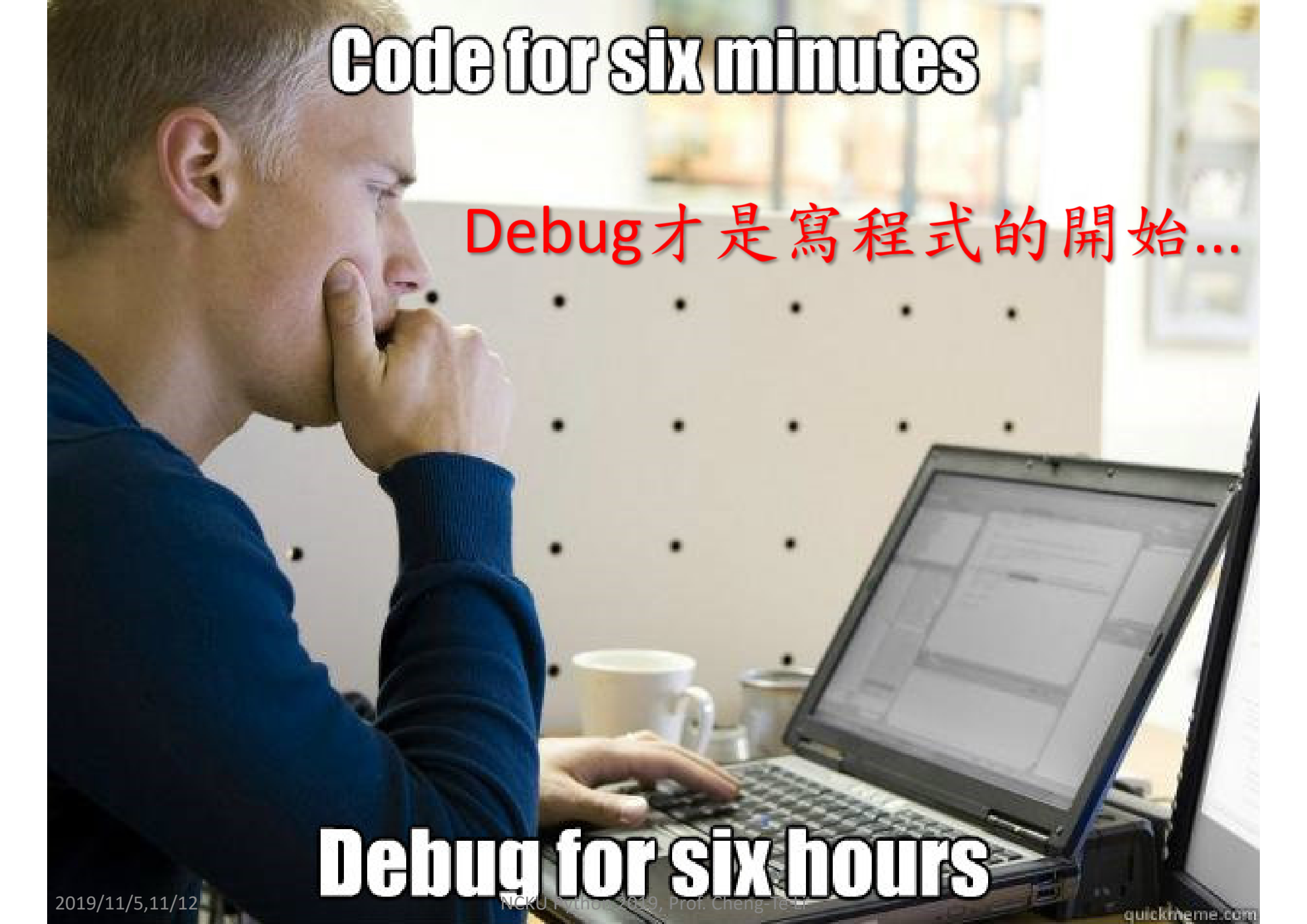
In-Class Exercise: Standard Deviation

- Use **list** and **while** loop to write a program (sd.py) to compute the value of **Standard Deviation** (標準差) for a list of integers

```
1 v = [82, 74, 80, 78, 86]
2 mean = sum(v)/len(v)
3 i = 0
4 diff2 = 0
5 while i < len(v):
6     tmp = (v[i]-mean) ** 2
7     diff2 = diff2 + tmp
8     i = i + 1
9 result = (diff2/(len(v)-1))**0.5
10 print("SD = %.2f" % result)
```

$$SD = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{X})^2}{n - 1}}$$

$$SD = 4.47$$



Code for six minutes

Debug才是寫程式的開始...

Debug for six hours

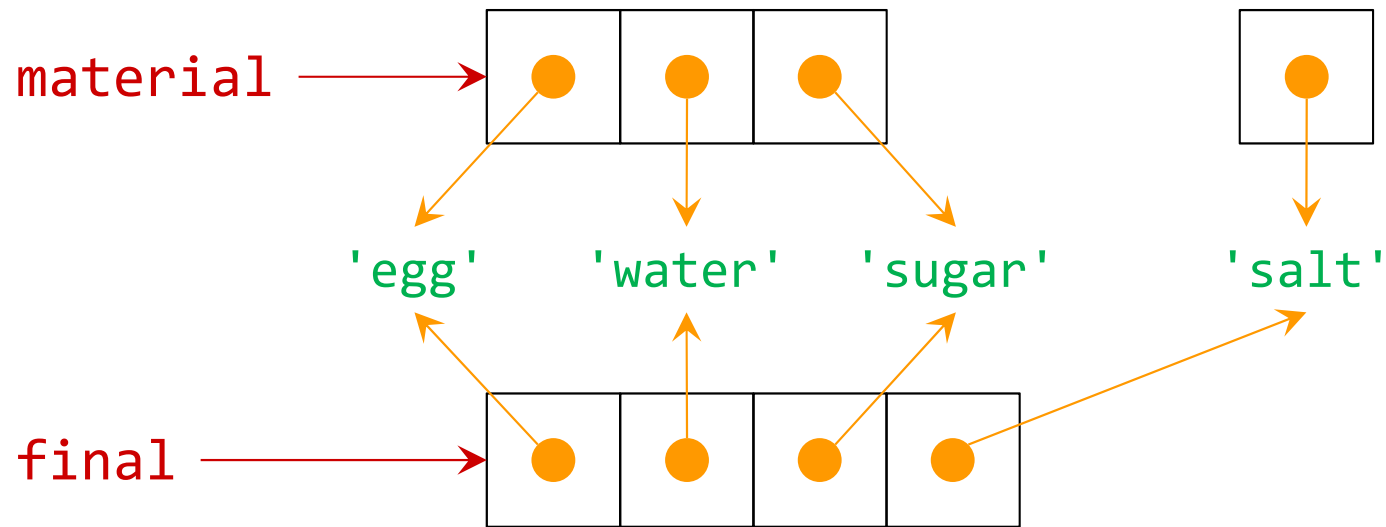
List Operations

```
material = ['egg', 'water', 'sugar']  
material + 'salt'
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: can only concatenate list (not "str") to list
```

- Python complains if we try to combine a list with other types in inappropriate ways
- It is not possible to append a **string** to a **list** with the **+** operator (recall different types cannot be +)
- But the **+** operator is **overloaded** for lists, so we can **concatenate lists** just like we concatenated strings

Memory Model for List Concatenation



```
material = ['egg', 'water', 'sugar']
final = material + ['salt']
print(final)      # ['egg', 'water', 'sugar', 'salt']

# * is for duplication (very commonly used for variable initialization)
list = [0] * 3    # duplicate the list up to 3 times
print(list)       # [0, 0, 0]
print(material*2) # ['egg', 'water', 'sugar', 'egg', 'water', 'sugar']
```

List Concatenation and Repeating

```
# you can assign lists
```

```
nums = [2, 3, 5]
```

```
morenums = [17]
```

```
# you can concatenate (+) or repeat (*) lists
```

```
n1 = 3 * morenums
```

```
print(n1)          # [17, 17, 17]
```

```
n2 = nums + n1
```

```
print(n2)          # [2, 3, 5, 17, 17, 17]
```

```
# you can build up a list by concatenation
```

```
n3 = []            # initialize
```

```
n3 = n3 + 101      TypeError: can only concatenate list (not "int") to list
```

```
n3 = n3 + [101]
```

```
n3 = n3 + [143]
```

```
print(n3)          # [101, 143]
```

List Equality Testing

- Use the `==` and `!=` operators to compare whether or not two lists **have the same elements** and **in the same order**

```
n1, n2, n3 = [1, 5, 9], [2, 3], [5, 1, 9]
print(n1 == n2)           # False
print(n1 == [1, 5, 9])    # True
print(n1 == n3)           # False
print(n1 != n2)           # True
```

```
notes = ["Do", "Re", "Mi"]
print(notes == ["Mi", "Re", "Do"]) # False
print(notes != ["Mi", "Re", "Do"]) # True
print(notes[2] == ["Mi"])          # False
print(notes[2] == "Mi")             # True
```

List Functions and Methods

```
1 + 2 + 3 # 6
```

```
sum([1, 2, 3]) # 6
```

```
sum(['a', 'b', 'c'])
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

```
music = 'Do Re Mi'.split()
```

```
print(music) # ['Do', 'Re', 'Mi']
```

```
new_music = music * 3 # applying * to lists indicates duplication
```

```
print(new_music) # ['Do', 'Re', 'Mi', 'Do', 'Re', 'Mi', 'Do', 'Re', 'Mi']
```

- `sum()` requires its **argument list** to contain numbers
- Note how we use the `split` method of a string to turn the **string** 'Do Re Mi' into a **three-element list** ['Do', 'Re', 'Mi']

Example

- Write a program that takes a numeric list and a "splitter" value. Then partitions the list into those smaller or equal to than the splitter and those larger. Sum up each part

```
1 splitter = 5
2 lst = [6, 9, 5, 2, 0, 7, 3, 4, 8, 1]
3 small = []
4 large = []
5 i = 0
6 while i < len(lst):
7     if lst[i] <= splitter:
8         small = small + [lst[i]]
9     else:
10        large = large + [lst[i]]
11    i = i + 1
12 print("Small Sum:", sum(small))
13 print("Large Sum:", sum(large))
```

Small Sum:	15
Large Sum:	30

Slicing Lists

- We can create a **new list** from an existing list by slicing
- For list, `list[i:j]` is the **slice** of the original list, from index **i** (inclusive) to index **j** (exclusive)

Slice with [*start* : *end* : *step*]

[:] extracts the entire sequence from start to end

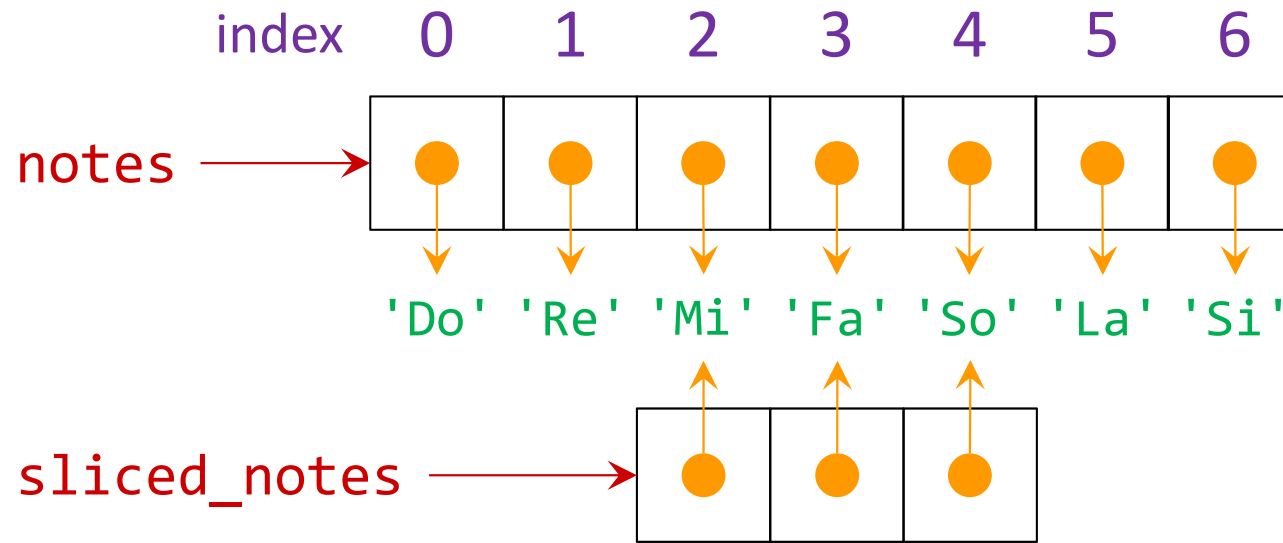
[*start* :] specifies from the *start* offset to the end

[: *end*] specifies from the beginning to the *end* offset **minus 1**

[*start* : *end*] indicates from the *start* offset to the *end* offset **minus 1**

[*start* : *end* : *step*] extracts from the *start* offset to the *end* offset **minus 1**, skipping characters by *step*

Slicing Memory Model



```
notes = ["Do", "Re", "Mi", "Fa", "So", "La", "Si"]
notes[1:5]          # ['Re', 'Mi', 'Fa', 'So']
sliced_notes = notes[2:5] # slicing
print(sliced_notes)   # ['Mi', 'Fa', 'So']
```

- Slicing does not modify lists
- That means: Slicing leaves the original list intact
- **Slicing returns a new list!!** (←very important)

Slicing Strings

	"Do"	"Re"	"Mi"	"Fa"	"So"	"La"	"Si"
index	0	1	2	3	4	5	6

```
notes = ["Do", "Re", "Mi", "Fa", "So", "La", "Si"]
print(notes[4])           # So
print(notes[:])           # ["Do", "Re", "Mi", "Fa", "So", "La", "Si"]
print(notes[2:5])         # ['Mi', 'Fa', 'So']
print(notes[1:4])         # ['Re', 'Mi', 'Fa']
print(notes[3:])          # ['Fa', 'So', 'La', 'Si']
print(notes[:5])          # ['Do', 'Re', 'Mi', 'Fa', 'So']
print(notes[1:6:2])       # ['Re', 'Fa', 'La']
print(notes[1::3])        # ['Re', 'So']
print(notes[::2])         # ['Do', 'Mi', 'So', 'Si']
print(notes[2][0])        # M
```


List Copying vs. List Aliasing

```
notes = ["Do", "Re", "Mi", "Fa", "So", "La", "Si"]  
his_notes = notes[:]  
notes[2] = "XDD"  
print(notes)           # ["Do", "Re", "XDD", "Fa", "So", "La", "Si"]  
print(his_notes)       # ["Do", "Re", "Mi", "Fa", "So", "La", "Si"]
```

List Copying

```
notes = ["Do", "Re", "Mi", "Fa", "So", "La", "Si"]  
her_notes = notes  
notes[2] = "XDD"  
print(notes)           # ["Do", "Re", "XDD", "Fa", "So", "La", "Si"]  
print(her_notes)       # ["Do", "Re", "XDD", "Fa", "So", "La", "Si"]
```

List Aliasing

List Copying vs. List Aliasing

- List **Copying**
 - List copying is based on list slicing
 - To obtain a copy of the original
 - Remember **slicing returns a new list**
- List **Aliasing**
 - An alias is an alternative for something
 - In Python, two variables are said be **aliased if they refer to the same object**, that is, contain the same reference
 - If two variables contain a reference to the same list, **modifying the list using one variable will be “seen” by the other variable**

List Copying vs. List Aliasing

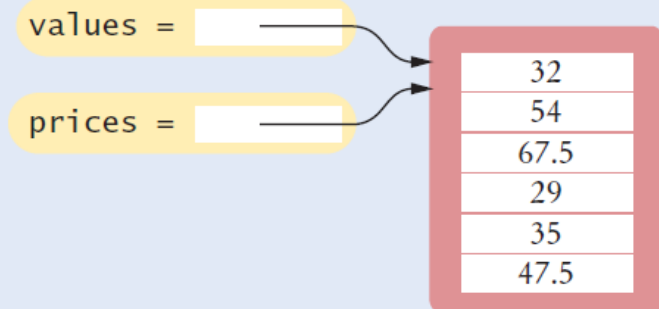
- `list(sequence)`: Create a new list containing all elements of the sequence

```
values = [67, 29, 35, 95]
prices = values # list aliasing
prices[2] = 75
print(values) # [67, 29, 75, 95]
print(prices) # [67, 29, 75, 95]
```

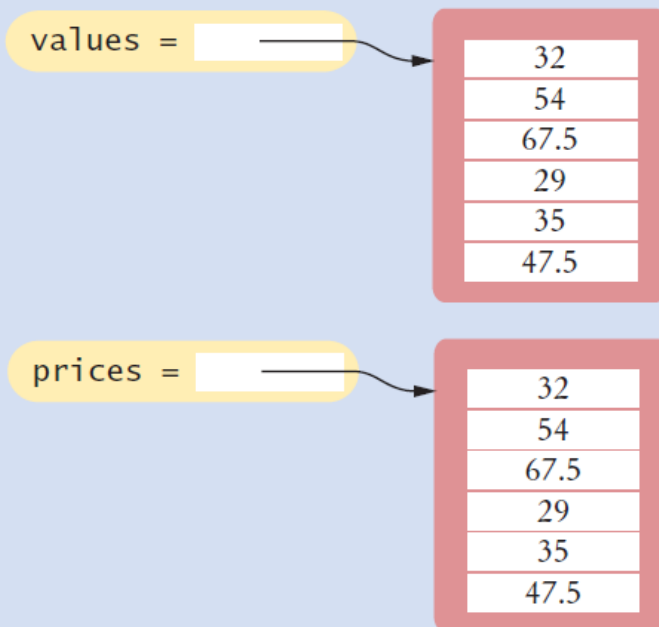
```
values = [67, 29, 35, 95]
prices = list(values) # list copying
prices[2] = 75
print(values) # [67, 29, 35, 95]
print(prices) # [67, 29, 75, 95]
```

```
text = "hello"
slist = list(text) # create new list
print(slist) # ['h', 'e', 'l', 'l', 'o']
```

1 After the assignment `prices = values`



2 After calling `prices = list(values)`



Example: Reverse the Elements in a List

```
1 mylist = ["a", "b", "c", "d", "e"]
2 reverse_list = mylist[:] # copy mylist to another list
3 n = len(mylist) # get the number of elements in mylist
4 i = 1
5 while i <= n:
6     # put the reversed elements in reverse_list
7     reverse_list[i-1] = mylist[n-i]
8     i = i + 1
9 print(reverse_list)
```

```
['e', 'd', 'c', 'b', 'a']
```

```
Print(mylist[::-1])
```

In-class Exercise:

Generate All Sublists of a List

```
1 L = ["a", "b", "c", "d", "e"]
2 i = 0
3 while i < len(L):
4     j = i + 1
5     while j <= len(L):
6         print(L[i:j])
7         j = j + 1
8     i = i + 1
```

```
['a']
['a', 'b']
['a', 'b', 'c']
['a', 'b', 'c', 'd']
['a', 'b', 'c', 'd', 'e']
['b']
['b', 'c']
['b', 'c', 'd']
['b', 'c', 'd', 'e']
['c']
['c', 'd']
['c', 'd', 'e']
['d']
['d', 'e']
['e']
```

Example: Sum of Neighbors in a List

```
1 s = input("Enter a positive integer: ")
2 # convert the input integer to a list of numbers
3 l = []
4 i = 0
5 while i < len(s):
6     l = l + [int(s[i])]
7     i = i + 1
8 print(l)
```

Goal: Input a positive integer.
Sum up neighbors of each number.
Output a new list for the result

```
9
10 # create a new list to store the result
11 newlist = []
12 i = 0
13 while i < len(l):
14     if i == 0:                # for the first element
15         newlist = newlist + [sum(l[0:2])]
16     elif i == len(l)-1:      # for the last element
17         newlist = newlist + [sum(l[len(l)-2:len(l)])]
18     else:                    # for other elements
19         newlist = newlist + [sum(l[i-1:i+2])]
20     i = i + 1
21 print(newlist)
```

```
c:\Python35-32\workspace>python neighbor_sum.py
Enter a positive integer: 18364521
[1, 8, 3, 6, 4, 5, 2, 1]
[9, 12, 17, 13, 15, 11, 8, 3]
```

Check Prime Number

- A number (>1) is **prime** (質數) if it has no divisors other than itself and one
 - ▣ **Primes**: 2, 3, 5, 7, 11, 13, ...
- Idea: try divisors 2, 3, ..., $n-1$ in turn to see if any can divide n evenly (整除與否), if none, n is prime

```
1 n = int(input("Please enter an integer (>1): "))
2 isPrime = True
3 i = 2
4 while i < n:
5     if n % i == 0:
6         isPrime = False
7         break
8     i = i + 1
9
10 if isPrime:
11     print("%d is a Prime number." % (n))
12 else:
13     print("%d is a not Prime number." % (n))
```

In-Class Exercise: Generate a List of Prime Number

- Write a program (list_prime.py)
 - Allow the user to input an positive integer **n**
 - Find and print the **list** of prime numbers from 2 to **n**

```
1 n = int(input("Enter a positive integer: "))
2 primelist = [] # the list to store prime numbers
3 candidate = 2 # test prime numbers from 2 to n
4 while candidate < n:
5     isPrime = True
6     i = 2
7     while i < candidate:
8         if candidate % i == 0:
9             isPrime = False
10            break
11            i = i + 1
12    if isPrime:
13        primelist = primelist + [candidate]
14        candidate = candidate + 1
15 print(primelist)
```

Check Prime Number

Built-in Functions for Lists

Method	Description
<code>L.append(v)</code>	Appends value <code>v</code> to list <code>L</code> , i.e. <code>L[len(L):] = [v]</code>
<code>L.extend(M)</code>	Appends all items in list <code>M</code> to the end of list <code>L</code> , i.e. <code>L[len(L):] = M</code>
<code>L.index(v)</code> 取得索引	Returns the index of the first item whose value is <code>v</code>
<code>L.insert(i, v)</code> 插隊	Inserts value <code>v</code> at index <code>i</code> in list <code>L</code> , shifting following items to make room
<code>L.pop()</code> 拿出最後一個元素	Removes and returns the last element of <code>L</code> , which must be nonempty; <code>L.pop(i)</code> removes and returns the item at index <code>i</code>
<code>L.remove(v)</code> 移除	Removes the first occurrence of value <code>v</code> from list <code>L</code>
★ <code>L.reverse()</code> 倒轉	Reverses the order of the values in list <code>L</code>
★ <code>L.sort()</code> 排序	Sorts the values in list <code>L</code> in ascending order
<code>L.sort(reverse=True)</code>	Sorts the values in list <code>L</code> in descending order

Modifying Lists

```
x = [1, 2, 3, 4]
x[1] = "two"
print(x)                # [1, 'two', 3, 4]
```

```
x = [1, 2, 3, 4]
x[len(x):] = [5, 6, 7]  # append list to the end of list
print(x)                # [1, 2, 3, 4, 5, 6, 7]
x[:0] = [-1, 0]         # append list to the front of list
print(x)                # [-1, 0, 1, 2, 3, 4, 5, 6, 7]
x[1:-1] = []            # remove elements from list
print(x)                # [-1, 7]
```

```
x = [1, 2, 3, 4]
x.append("five")        # append an element to the end of list
print(x)                # [1, 2, 3, 4, 'five']
```

Modifying Lists

```
x = [1, 2, 3, 4]
```

```
y = [5, 6, 7]
```

```
x.append(y)
```

```
print(x)
```

```
# append a list to the end of list
```

```
# [1, 2, 3, 4, [5, 6, 7]]
```

```
x = [1, 2, 3, 4]
```

```
y = [5, 6, 7]
```

```
x.extend(y)
```

```
print(x)
```

```
# extend a list by another list
```

```
# [1, 2, 3, 4, 5, 6, 7]
```

```
x = [1, 2, 3]
```

```
x.insert(2, "hello")
```

```
print(x)
```

```
x.insert(0, "start")
```

```
print(x)
```

```
# insert an element to a position
```

```
# [1, 2, 'hello', 3]
```

```
# ['start', 1, 2, 'hello', 3]
```

```
a = x.pop()
```

```
print(a)
```

```
print(x)
```

```
print(x.pop())
```

```
print(x)
```

```
# 3 is removed and returned, assigned to a
```

```
# 3 is printed
```

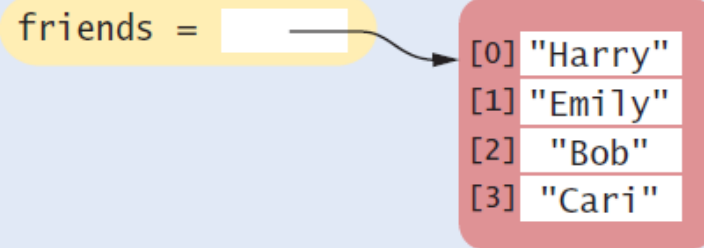
```
# ['start', 1, 2, 'hello']
```

```
# 'hello' is removed and returned, printed
```

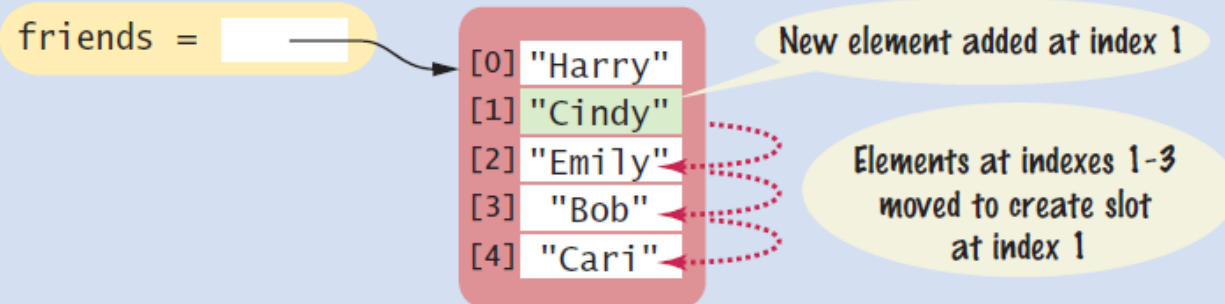
```
# ['start', 1, 2]
```

```
friends = ["Harry", "Emily", "Bob", "Cari"]  
friends.insert(1, "Cindy")  
friends.insert(5, "Bill")
```

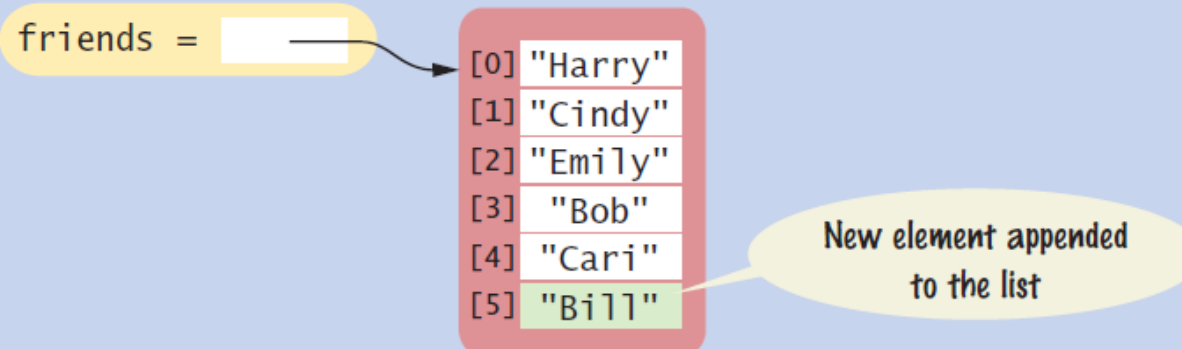
1 The newly created list



2 `friends.insert(1, "Cindy")`



3 `friends.insert(5, "Bill")`



Example

- Read input from a user and store it in a list for later processing
- **Start with an empty list** (`values=[]`)
- As each value is read, **append the value to the end of the list**

```
Please enter values, Q to quit:
3
11
5
-2
7
Q
-----
3
11 <= largest value
5
-2
7
```

```
1 # create an empty list
2 values = []
3
4 # read the input values
5 print("Please enter values, Q to quit:")
6 userInput = input("")
7 while userInput.upper() != "Q" :
8     values.append(int(userInput))
9     userInput = input("")
10
11 # Find the largest value.
12 largest = values[0]
13 i = 0
14 while i < len(values):
15     if values[i] > largest :
16         largest = values[i]
17     i = i + 1
18 print("-----")
19
20 # Print all values, marking the largest.
21 i = 0
22 while i < len(values):
23     print(values[i], end="")
24     if values[i] == largest :
25         print(" <= largest value", end="")
26     print()
27     i = i + 1
```

Modifying Lists

```
x = ['a', 2, 'c', 7, 9, 11]
del x[1]          # delete an item in a list
print(x)          # ['a', 'c', 7, 9, 11]
```

```
del x[:2]         # delete a slice in a list
print(x)          # [7, 9, 11]
```

```
x = [1, 2, 3, 4, 3, 5]
x.remove(3)        # remove() looks for the first instance of a given
                  # value in a list and removes that value from the list
print(x)           # [1, 2, 4, 3, 5]
x.remove(3)
print(x)           # [1, 2, 4, 5]
```

```
x = [1, 3, 5, 7, 9]
x.reverse()        # reverse a list in place, equal to x[::-1]
print(x)           # [9, 7, 5, 3, 1]
```

Sorting Lists

```
x = [3, 8, 4, 0, 2, 1]
```

```
x.sort() # a list can be sorted using the built-in sort()
```

```
print(x) # [0, 1, 2, 3, 4, 8]
```

```
x = [2, 4, 1, 3]
```

```
y = x[:] # copy a list using slicing
```

```
y.sort()
```

```
print("x:", x, "vs. y:", y) # x: [2, 4, 1, 3] vs. y: [1, 2, 3, 4]
```

```
x = [2, 4, 1, 3]
```

```
y = x[:] # copy a list using slicing
```

```
z = y.sort() # nothing is assigned to z
```

sort() does not return a value
but modifies the list itself!

```
x = [2, 4, 1, 3]
```

```
y = sorted(x)
```

```
print("x:", x, "vs. y:", y) # x: [2, 4, 1, 3] vs. y: [1, 2, 3, 4]
```

```
# sorted() returns a sorted list from any iterable object
```

```
x = [2, 4, 1, 3]
```

```
y = sorted(x, reverse = True) # sort in a descending order
```

```
print("x:", x, "vs. y:", y) # x: [2, 4, 1, 3] vs. y: [4, 3, 2, 1]
```

Sorting Lists

```
x = ["Life", "is", "Enchanting"]  
x.sort()  
print(x)                # ['Enchanting', 'Life', 'is']
```

```
x = [1, 2, 'hello', 3]  
x.sort()  
print(x)
```

Traceback (most recent call last):
File "<stdin>", line 1, in <module>
TypeError: unorderable types: str() < int()

```
x = [[3, 5], [2, 9], [2, 3], [4, 1], [3, 2]] # a list of lists  
x.sort()  
print(x)                # [[2, 3], [2, 9], [3, 2], [3, 5], [4, 1]]  
# the sublists are sorted first by ascending first element  
# and then by ascending second element, so on so forth
```


Recall: Built-in Functions for Strings

- **split()**: Break a string into a list of strings according to a given separator

```
line = "Hillary Clinton clarified her misleading statements."
line2 = line.split(" ")
print(line2)
# ['Hillary', 'Clinton', 'clarified', 'her', 'misleading',
# 'statements.']
myDate = "2016-09-27"
myDate.split("-")      # ['2016', '09', '27']
myDate.split(0)        # error
myDate.split("0")     # ['2', '16-', '9-27']
```



```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: Can't convert 'int' object to str implicitly
```

Recall: Built-in Functions for Strings

- **join()**: **Collapse** a **list of strings** into a single string according to a given **separator**

```
presidents = ['Bush', 'Clinton', 'Bush', 'Obama']
result = "->".join(presidents)
print(result)      # Bush->Clinton->Bush->Obama

line = "Hillary Clinton clarified her misleading statements."
wlist = line.split(" ")
result = "".join(wlist)
print(result)
# HillaryClintonclarifiedhermisleadingstatements.
result = ".".join(wlist)
print(result)
# Hillary.Clinton.clarified.her.misleading.statements.
```

Example: Convert a List to a Set

- Write a program that allows the user to input a list of numbers (separated by a whitespace), then remove duplicated numbers so that it becomes a set

```
1 s = input("Enter a list of numbers: ")
2 nlist = s.split(" ")
3 t = sorted(nlist) # 1 1 2 2 2 3 4 4 7 7 7
4 now = t[0]
5 i = 1
6 while i < len(t):
7     if t[i] == now:
8         nlist.remove(t[i])
9     else:
10        now = t[i]
11        i = i + 1
12 result = " ".join(nlist)
13 print(result)
```

Input & Output

```
c:\Python35-32\workspace>python list2set.py
Enter a list of numbers: 7 4 2 1 2 4 7 7 2 1 3
4 7 2 1 3
```

Another Solution

```
1 s = input("Enter a list of numbers: ")
2 nlist = s.split(" ")
3 print(set(nlist))
4 print(" ".join(set(nlist)))
```

```
Enter a list of numbers: 7 4 2 1 2 4 7 7 2 1 3
{'3', '4', '1', '2', '7'}
3 4 1 2 7
```

The Range Function

- Use the `range` function to generate a **list** of **integer** values in a range; it works only for **ints**
- `range()` can return an **iterable object**
- `list()` is used only to force the items range would generate to appear as a list
 - It's not normally used in actual code

`range(stop)`

or

`range(start, stop, step)`
 一定要有 一定要有 可有可無

```
print(range(0, 5))           # range(0, 5)
type(range(0, 5))           # <class 'range'>
list(range(0, 5))            # [0, 1, 2, 3, 4]
print(list(range(0, 5)))     # [0, 1, 2, 3, 4]
list(range(3))                # [0, 1, 2]
list(range(2, 7))            # [2, 3, 4, 5, 6]
list(range(10, 25, 3))       # [10, 13, 16, 19, 22]
```

List Membership with the **in** Operator

- Test if a value is in a list using the **in** operator, which returns a **Boolean** value (**True** or **False**)
- You can also use the converse, the **not in** operator

```
3 in [1, 3, 4, 5]                # True
3 not in [1, 3, 4, 5]            # False
3 in ["one", "two", "three"]     # False
3 not in ["one", "two", "three"] # True
'two' not in ["one", "two", "three"] # False
'Two' not in ["one", "two", "three"] # True
3 in range(3)                    # False
3 in range(1, 5)                 # True
```

List Search with `index()`, List Match with `count()`

```
# index() search through a list looking for an element equivalent  
# to a given value, and return the position of that element
```

```
x = [1, 3, "five", 7, -2]
```

```
x.index(7)      # 3
```

```
x.index(5)
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
ValueError: 5 is not in list
```

```
if 5 in x:
```

```
    print("5\'s position is:", x.index(5))
```

```
else:
```

```
    print("5 is not in x")
```

```
# count() also searches through a list, looking for a given value,  
# but it returns the number of times that value is found in the  
# list rather than positional information
```

```
x = [1, 2, 2, 3, 5, 2, 5]
```

```
x.count(2)      # 3
```

```
x.count(5)      # 2
```

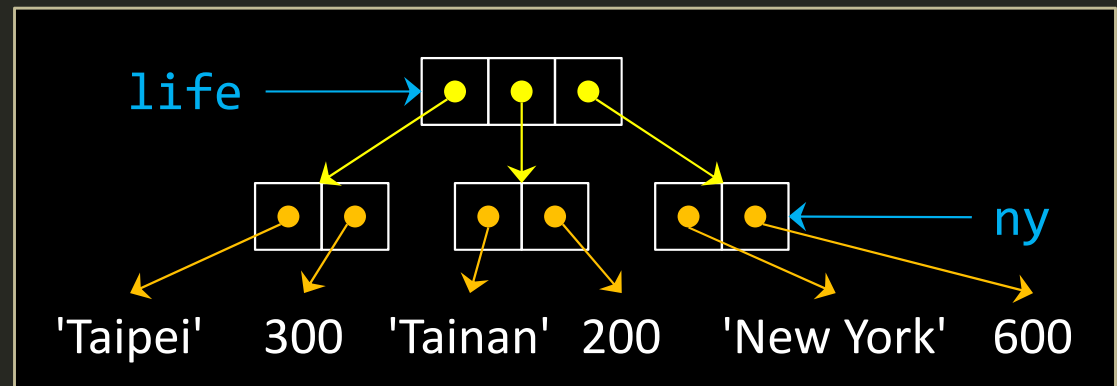
```
x.count(4)      # 0
```

Nested Lists

- Since lists are heterogeneous, they can contain other lists
- We can also assign sublists to variables

```
life = [['Taipei', 300], ['Tainan', 200], ['New York', 500]]
life[0]      # ['Taipei', 300]
print("Life expectancy in", life[1][0], "is", life[1][1])
      # Life expectancy in Tainan is 200
```

```
ny = life[2] # aliasing
print(ny[0]) # New York
print(ny[1]) # 500
ny[1] = 600
print(life)
# [['Taipei', 300], ['Tainan', 200], ['New York', 600]]
```



Example: A World Population Table

```
c:\Python35-32\workspace>python world_population.py
Year 1750 1800 1850 1900 1950 2000 2050
Africa 106 107 111 133 221 767 1766
Asia 502 635 809 947 1402 3634 5268
Australia 2 2 2 6 13 30 46
Europe 163 203 276 408 547 729 628
North America 2 7 26 82 172 307 392
South America 16 24 38 74 167 511 809
World 791 978 1262 1650 2522 5978 8909
```

Problem Statement Consider the following population data.

Population Per Continent (in millions)							
Year	1750	1800	1850	1900	1950	2000	2050
Africa	106	107	111	133	221	767	1766
Asia	502	635	809	947	1402	3634	5268
Australia	2	2	2	6	13	30	46
Europe	163	203	276	408	547	729	628
North America	2	7	26	82	172	307	392
South America	16	24	38	74	167	511	809

You are to print the data in tabular format and add column totals that show the total world populations in the given years.

Example: A World Population Table (1/2)

```
1 ROWS, COLUMNS = 6, 7
2
3 # initialize the populations table
4 pop = [ [106, 107, 111, 133, 221, 767, 1766],
5         [502, 635, 809, 947, 1402, 3634, 5268],
6         [2, 2, 2, 6, 13, 30, 46],
7         [163, 203, 276, 408, 547, 729, 628],
8         [2, 7, 26, 82, 172, 307, 392],
9         [16, 24, 38, 74, 167, 511, 809] ]
10
11 # define a list of continent names
12 continents = ["Africa", "Asia", "Australia",
13              "Europe", "North America", "South America"]
14
15 ytotal = []
16 i = 0
17 while i < COLUMNS:
18     ytotal.append(0)
19     i = i + 1
20
21 # print the table header
22 print("%48s" % ("Year 1750 1800 1850 1900 1950 2000 2050"))
```

```
c:\Python35-32\workspace>python world_population.py
Year 1750 1800 1850 1900 1950 2000 2050
Africa 106 107 111 133 221 767 1766
Asia 502 635 809 947 1402 3634 5268
Australia 2 2 2 6 13 30 46
Europe 163 203 276 408 547 729 628
North America 2 7 26 82 172 307 392
South America 16 24 38 74 167 511 809
World 791 978 1262 1650 2522 5978 8909
```

Example: A World Population Table (2/2)

```
23 # print population data
24 i = 0
25 while i < ROWS:
26     # print the i-th row
27     print("%13s" % continents[i], end="")
28     j = 0
29     while j < COLUMNS:
30         print("%5d" % pop[i][j], end="")
31         ytotal[j] += pop[i][j]
32         j = j + 1
33     print()
34     i = i + 1
35
36 # print column totals
37 print("%13s" % ("World"), end="")
38 j = 0
39 while j < COLUMNS:
40     print("%5d" % ytotal[j], end="")
41     j = j + 1
42 print()
```

```
c:\Python35-32\workspace>python world_population.py
Year 1750 1800 1850 1900 1950 2000 2050
Africa 106 107 111 133 221 767 1766
Asia 502 635 809 947 1402 3634 5268
Australia 2 2 2 6 13 30 46
Europe 163 203 276 408 547 729 628
North America 2 7 26 82 172 307 392
South America 16 24 38 74 167 511 809
World 791 978 1262 1650 2522 5978 8909
```

Example: Check Magic Square

- An $n \times n$ matrix that is filled with the numbers is a Magic Square if the sum of the elements in each row, in each column, and in the two diagonals is the same value
- Write a program that reads in 16 values from the keyboard and tests whether they form a magic square when put into a $n \times n$ table

16	3	2	13
5	10	11	8
9	6	7	12
4	15	14	1

Example: Check Magic Square (1/2)

```
1  #matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
2  matrix = [[4, 9, 2], [3, 5, 7], [8, 1, 6]]
3  #matrix = [[16, 23, 17], [78, 32, 21], [17, 16, 15]]
4  #matrix = [[23, 28, 21], [22, 24, 26], [27, 20, 25]]
5  #matrix = [[7, 12, 1, 14], [2, 13, 8, 11], [16, 3, 10, 5], [9, 6, 15, 4]]
6
7  dim = len(matrix[0])
8  sum_list = []
9
10 # Row sum
11 i = 0
12 while i < dim:
13     sum_list.append(sum(matrix[i]))
14     i = i + 1
15
16 # Column sum
17 j = 0
18 colsum = [0] * len(matrix) # initialize a list of zeros
19 while j < dim:
20     i = 0
21     while i < dim:
22         colsum[j] += matrix[i][j]
23         i = i + 1
24     j = j + 1
25 sum_list.extend(colsum)
```

16	3	2	13
5	10	11	8
9	6	7	12
4	15	14	1

Example: Check Magic Square (2/2)

```
26
27 # Diagonal sum (left-top to right-bottom)
28 i = 0
29 dlsum = 0
30 while i < dim:
31     dlsum += matrix[i][i]
32     i = i + 1
33 sum_list.append(dlsum)
34
35 # Diagonal sum (left-bottom to right-top)
36 i = 0
37 drsum = 0
38 diag_indices = range(dim-1, -1, -1)
39 while i < len(diag_indices):
40     drsum += matrix[diag_indices[i]][i]
41     i = i + 1
42 sum_list.append(drsum)
43
44 print(sum_list)
45
46 # convert the sum_list to a set
47 if len(set(sum_list)) > 1:
48     print("No, it is not a Magic Square!")
49 else:
50     print("Yes, it is a Magic Square!")
```

16	3	2	13
5	10	11	8
9	6	7	12
4	15	14	1

In-Class Exercise

- Given students' scores in three classes, contained by a nested list, by considering only scores greater than 60, report the highest and the lowest three scores

Hints: if, while, len(), extend(), reverse(), index(), sort() or sorted(), del, slicing

```
1  L = [[100, 83, 88], [85, 33, 92, 89, 95], [91, 84, 86, 87, 96, 58]]
2  new_list = []
3  i = 0
4  while i < len(L):
5      j = 0
6      while j < len(L[i]):
7          if L[i][j] < 60:
8              del L[i][j]
9              j = j + 1
10     new_list.extend(L[i])
11     i = i + 1
12 sorted_list = sorted(new_list)
13 sorted_list.reverse()
14 print("The highest three:", sorted_list[:3])
15 print("The lowest three:", sorted_list[len(sorted_list)-3:])
16 print("91 is ranked at", sorted_list.index(91)+1, "position")
```

The highest three: [100, 96, 95]
The lowest three: [85, 84, 83]
91 is ranked at 5 position

You Must Know!

- Initialization:

Always initialize your variables!

- Lists are Mutable:

If you perform an operation on a list, it changes the list. In contrast, strings are immutable

- Copying of Lists:

Don't simply copy a list by variable assignment because this makes them the same object

Copy vs. Aliasing

- `id(name)`: see the object's (starting) address in memory

```
values = [67, 29, 35, 95]
prices = values                # list aliasing
prices[2] = 75
print(values, prices)          # [67, 29, 75, 95] [67, 29, 75, 95]
print(id(values), id(prices))  # 17701424 17701424
```

```
values = [67, 29, 35, 95]
prices = values[:]             # list copying
prices[2] = 75
print(values, prices)          # [67, 29, 35, 95] [67, 29, 75, 95]
print(id(values), id(prices))  # 17701304 17701904
```

```
values = [67, 29, 35, 95]
prices = list(values)          # list copying
prices[2] = 75
print(values, prices)          # [67, 29, 35, 95] [67, 29, 75, 95]
print(id(values), id(prices))  # 17701424 17719424
```


Copy vs. Aliasing

```
a = 1
b = a
print(id(a), id(b))      # 1532188944 1532188944
b = b + 1
print(a, b)              # 1 2
print(id(a), id(b))      # 1532188944 1532188960
```

```
a = [1, 2]
b = a[1]
print(id(a[1]), id(b))   # 1532188960 1532188960
b = b + 1
print(a[1], b)           # 2 3
print(id(a[1]), id(b))   # 1532188960 1532188976
```

```
a = [[1, 2], [3, 4]]
b = a[1]
print(id(a[1]), id(b))   # 34544056 34544056
b = b + [1]
print(a[1], b)           # [3, 4] [3, 4, 1]
print(id(a[1]), id(b))   # 34544056 34543936
```

More Examples

```
a = [1, 2]
b = a[0]
b = b + 1
print(a[0]) # 1
```

```
a = [1, 2]
a[0] = a[0] + 1
print(a[0]) # 2
```

```
a = [[1, 2], [3, 4]]
b = a[0]
b = b + [1]
print(a[0]) # [1, 2]
```

```
a = [1, 2]
b = [a, a]
print(b) # [[1, 2], [1, 2]]
```

```
a = [1, 2] # id(a): 23010280
b = [a, a]
a.append(3) # id(a): 23010280
print(b)
# [[1, 2, 3], [1, 2, 3]]
```

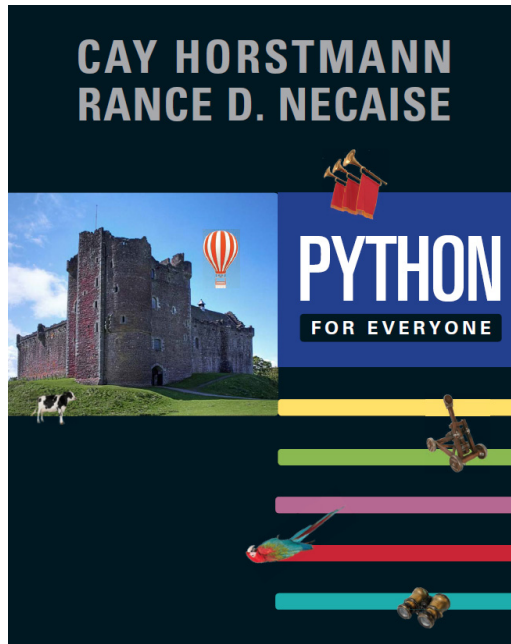
```
a = [1, 2] # id(a): 24058856
b = [a, a]
a = [3, 4] # id(a): 24058416
print(b) # [[1, 2], [1, 2]]
```

```
a = 'xy' # id(a): 21322080
b = [a, a]
a = 'xy'+'z' # id(a): 22062176
print(b)
# ['xy', 'xy']
```

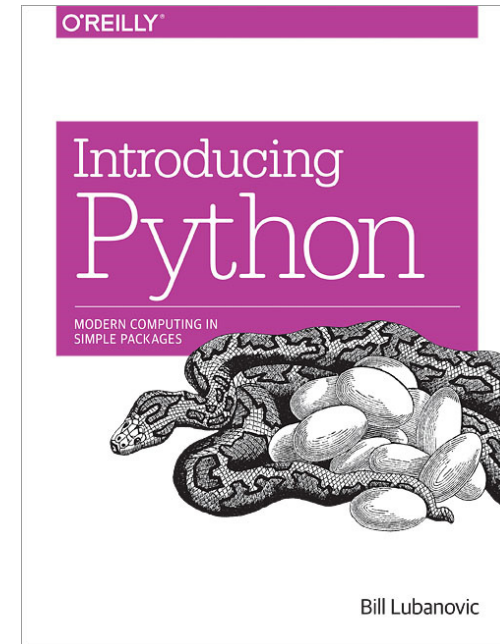
Summary

- List Indexing and Slicing
- List Operations
- List Copying vs. List Aliasing
- List Built-in Functions
- Nested Lists

Suggested Reading



P.277 – P.340



P.41 – P.50